

Tech Digest *with Sam.*

The week in AI, engineering & the quiet shifts reshaping how we build.



CURATED & WRITTEN BY
Sam Kirubaharan

INSIDE THIS ISSUE

WEEK 16 • THIRTEEN STORIES

01	● BYLINE	Harness Engineering: The New Frontier of Agentic Design	02
02	● TRENDING	Anthropic Just Dropped Opus 4.7 – and It Verifies Its Own Work	03
03	● LOVE IT!	Claude Design Is Here – and I'm Using It	04
04	● REFLECT	When AI Works Exactly as Designed	05
05	● INCOMING	Anthropic's Three Drops and What They Signal	06
06	● BYLINE	Knowledge That Compounds	07
07	● MR & MS CURIOUS	RAG Is Dead. Or Is It?	08
08	● TRENDING	Agent Tool Design Is More Art Than Science	10
09	● LOVE IT!	Caveman: 75% Fewer Tokens, Same Signal	11
10	● BYLINE	My Dev Setup, Now with Claude	12
11	● INCOMING	The Advisor-Executor Pattern	13
12	● TRENDING	LangWatch Just Open-Sourced Adversarial Testing	14
13	● REFLECT	The Names Behind the AI Incidents	15

✍ BYLINE · THE ESSAY

Harness engineering: the new frontier of *agentic design*.

*There's a framing I've been sitting with lately that I think cuts closer to the truth about where AI engineering is heading than most takes out there: **the real frontier isn't writing better LLM prompts – it's designing the harness around the model.***

I published a piece on this recently, and the core idea is that Agentic Harnessing is genuinely hard in a way that software engineering is quickly ceasing to be. LLMs are already close to "solved" for verifiable tasks – give them a spec and a test suite and they'll iterate toward it. But building the environment in which an agent reliably operates? That's still a deeply human problem, and it's one that AI can't yet bootstrap itself out of.

Here's what Harness Engineering actually involves:

- **Context Engineering** – deciding what information the agent sees and when, so it stays sharp and doesn't get polluted by noise
- **Feedback Loops** – feeding activity back to the agent to compound knowledge and prevent the same mistakes repeating
- **Advanced Tooling** – targeted scripts and MCPs that introduce determinism into an inherently non-deterministic model
- **Structured Paths** – defining explicit state transitions so an agent moves with intent instead of wandering

"The harness is what bridges the gap between a raw model and a sophisticated, reliable digital employee."

Software engineers are starting to feel like the hard part of their job is disappearing. What's emerging in its place is this discipline – part architect, part systems thinker – that I'm calling the Harness Engineer. There's no universal test suite for whether a harness is "good." That judgment still requires human intuition – and right now, not many people are deliberately building it.

FULL ESSAY ON MEDIUM

Harness Engineering: The New Frontier of Agentic Design ↗

TRENDING · THIS WEEK

Anthropic just dropped Opus 4.7 – and it *verifies its own work*.

There's no "later" anymore at Anthropic. Claude Opus 4.7 landed this week, and if you've been following the model race, this one has a few features worth actually paying attention to.

The headline isn't just benchmark scores. It's what's baked into the model itself:

- **Self-verification is now native** – Opus 4.7 checks its own outputs before reporting back, without you needing to prompt it to do so
- Agentic coding scores have jumped significantly – it's built for long-running tasks with less hand-holding
- Vision got a serious upgrade: 3× higher resolution, which means usable outputs for interfaces, slides, and docs
- Grounding techniques are now built into the model layer, not bolted on through tool calls
- Cybersecurity guardrails have been tightened (a miniature version of the Mythos framework)

The verification step is the thing I keep coming back to. Anthropic made a deliberate architectural decision to bake it into the model rather than into Claude Code or another tool layer. That's meaningful – it means the behavior travels with the model across every deployment, not just the ones where someone remembered to add the check.

That's the shift: from "prompt it to verify" to "it just does."

The grounding bullet is worth pausing on too. Each major AI platform has its own approach here – Google grounds via Google Search, Amazon grounds via Specification. This is Anthropic's way: grounding built into the model layer itself, not as an external tool call or integration. Same destination, very different architectural choices.

An idea worth exploring: what if model verification capabilities were linked natively to business verification schemes? The model already checks its own outputs – imagine that wired directly into your compliance rules, approval workflows, or domain-specific quality gates. Native verification meets business logic, without the glue code in between.

♥ LOVE IT! · SOMETHING I'M USING

Claude Design is here — and *I'm using it.*

Anthropic just dropped Claude Design and I have to talk about it — this one genuinely surprised me.

A dedicated tool for creating designs, prototypes, slides, and one-pagers entirely through conversation. No drag-and-drop. No template browsing. Just describe what you want, and Claude builds the first version.

Here's what makes this more than a cool demo:

- **Powered by Claude Opus 4.7 Vision** — actually understands visual context, not just text prompts
- **Reads your codebase and design files** to build your team's design system automatically — every output stays on-brand without manual style enforcement
- Iterate through conversation, inline comments, direct edits, or custom sliders
- Export to Canva, PDF, or PPTX — or hand off directly to Claude Code for implementation
- Available now in research preview on Pro, Max, Team, and Enterprise plans

The part I keep coming back to: it learns your design system from your actual files. That's the move that shifts this from novelty to real workflow change. Brand guidelines in a Figma file or CSS tokens? Claude internalises that and applies it consistently. Most design tools still can't do that.

We're watching the design handoff problem get quietly solved — and I absolutely love it.

Full disclosure: this newsletter is designed using Claude Design. I've been using it personally and it's been a genuinely great experience. Try it at claude.ai/design.

When AI works *exactly as designed*.

The question we keep asking about AI is: what happens when it goes wrong? Lauren Good asks a harder one: what happens when it goes exactly right?

In a white paper published this week, Lauren – Human Systems Architect and Licensed Clinical Consultant – examines two events that arrived just 13 days apart. A Los Angeles jury found Meta and Google liable for deliberately designing platforms to addict children. Days later, Anthropic disclosed it had withheld Claude Mythos Preview because of what it autonomously did to critical software infrastructure – while working as intended.

Neither system failed. Both reshaped the people inside them.

Lauren calls this the **Formation Effect** – what AI does to human judgment, competence, and autonomy during normal, correct, intended use. It operates through four dimensions:

- **Skill Foreclosure** – unaided competence quietly erodes through disuse, not failure
- **Behavioural Masking** – people adapt to what the system rewards, until the adaptation feels like identity
- **Oversight Fragility** – as trust in a system grows, human supervision thins – not through negligence, but through rational deference
- **Regenerative Agency** – can people still reach for self-direction, or has the system made that feel unnecessary?

Current governance frameworks – the EU AI Act, voluntary safety evaluations, jailbreak testing – are all built to catch what goes wrong. None of them measure what goes right and still quietly reshapes the humans operating inside it.

The effect AI and software products can create on us when they work just as expected – that's the part we're not yet measuring.

A CALL TO ACTION

Pause, reflect, and initiate conversations. Collaborate with AI governance experts. Let's push for a community-driven AI landscape.

[READ THE WHITE PAPER BY LAUREN GOOD →](#)

↓ INCOMING · SIGNALS

Anthropic's three drops — and what they *signal*.

Anthropic just dropped three things in close succession — and together they point at something bigger than any single release.

Here's what landed:

- **Claude Opus 4.7** — the new flagship model. Stronger agentic coding scores, built-in output verification before it reports back, 3× higher image resolution, and meaningfully better at following complex instructions
- **Claude Code Routines** — scheduled, API-triggered, and GitHub-triggered automation that spins up Claude Code Web instances autonomously. No machine needs to stay awake. No IDE required
- **Redesigned Claude desktop app** — rebuilt from the ground up for parallel work. Integrated terminal, in-app file editing, rebuilt diff viewer, side chats, SSH connections

*The throughline: Anthropic isn't building a better chatbot.
They're building a software development operating system.*

Routines is the one I keep thinking about. The old Schedules feature required your machine to be running. Routines runs entirely via Claude Code Web — so you can trigger nightly issue triage, PR reviews, alert triage, or cross-repo SDK ports without any local infra. That's IDE-less development becoming genuinely viable, not just a talking point.

And Opus 4.7 verifying its own outputs before surfacing results? That's not a small tweak — it's Anthropic baking epistemic discipline into the model itself rather than leaving it to prompt engineering.

At some point "AI coding assistant" stops being an accurate frame. We're already past it.

 BYLINE · FROM MY DESK

Knowledge that *compounds*.

Graphify is worth your attention – an extended version of Karpathy's knowledge workflow, now open source and runnable in a single command.

Karpathy had been manually piping papers, tweets, and screenshots into an LLM-compiled wiki, navigated through Obsidian. Knowledge that compounds every time you add something new. His closing line: *"I think there is room here for an incredible new product instead of a hacky collection of scripts."* Graphify shipped days later.

For real work though, I prefer building it bottom-up – starting from nothing and adding only the minimum required. That's what my KB skill set does. Three skills, one loop:

- **KB Gather** – pulls from JIRA tickets, web pages, documents (Word, Excel, PowerPoint), codebase, and chat/call transcripts including LLM conversations – and writes everything into a folder of markdown files
- **KB Consult** – attaches relevant KB topics to an active chat session for Q&A and context enrichment. Detaches when you're done
- **KB Digest** – takes new things learned during a session and writes them back into the KB

The magic is in running it iteratively on the same data. Each pass, the KB gets more accurate. Connections sharpen. Gaps fill in.

The LLM indexes the knowledge and renders only what's relevant when it's needed – no bloat, no noise. This is the core of what Karpathy was after: **knowledge that compounds**. Every ticket, every doc, every conversation makes the next one sharper.

Simple to build, lightweight to run – and surprisingly effective. The bigger frameworks have their place, but for a developer deep in daily work, this kind of minimal loop often does more than you'd expect from something so lean.

MR & MS CURIOUS

RAG is dead. *Or is it?*

SAM'S THOUGHTS

MS. CURIOUS

ok so someone just sent me this – "RAG is Dead or is it?" – honestly? I think it's dead

MR. CURIOUS

it's not dead, you just know Naive RAG. there are versions to this

MS. CURIOUS
versions?

MR. CURIOUS

yes – Advanced RAG, Modular RAG, Agentic RAG. each one fixes what the last one got wrong. retrieval got smarter, not obsolete

MS. CURIOUS

ok but here's the thing – 1M token context windows. why retrieve at all when you can just... put everything in?

MS. CURIOUS

the models are getting big enough to swallow entire codebases. the whole retrieval problem starts dissolving when the context window is that large

MR. CURIOUS

that's actually a fair point

MS. CURIOUS

1M tokens isn't the ceiling either. that space is moving fast. the architecture question becomes less "what do I retrieve" and more "what do I even need to store separately"

MR. CURIOUS

so maybe the versions don't matter as much if the window keeps growing

MS. CURIOUS

that's where I'd put my attention right now

Thanks to [Rajeev Sakhuja](#) for sparking this – see his [RAG Is Dead?](#) and [1M Context Window: Who Needs RAG?](#)

TRENDING · CRAFT

Agent tool design is *more art than science*.

There's a piece out from Thariq Shihpar on the Anthropic/Claude blog that I keep thinking about: Seeing Like an Agent: How We Design Tools in Claude Code.

The core thesis is deceptively simple – designing the tools that an AI agent uses is one of the hardest problems in building agents, and there's no formula for getting it right.

A few things that stood out:

- **Tool design is fundamentally iterative** – the Claude Code team didn't architect the perfect toolset upfront; they discovered it through cycles of building, observing, and adjusting
- The agent needs to "see" the world through its tools, which means poorly scoped tools don't just limit capability – they actively distort how the agent reasons
- Intuition plays a huge role; it's more like UX design than systems engineering

What strikes me is how this challenges the way most teams approach agent development. We spend a lot of time on model selection, prompt engineering, memory architecture – but the tooling layer often gets treated as an afterthought.

If the tools are the agent's senses, then bad tools don't just slow it down – they make it blind in ways it can't even detect.

Worth a read if you're building anything with agents. The full post is at claude.com/blog/seeing-like-an-agent. Most teams are still bolting on capabilities as they go. The ones treating tool design as a first-class discipline will build significantly better agents.

♥ LOVE IT! · AN EXPERIMENT

Caveman: 75% fewer tokens, same signal.

*Julius Brussee's **Caveman** project is one of those ideas that makes you stop and think – and I had to share it because I've felt this problem firsthand.*

Raw Claude Opus returns essays. Beautiful, thorough, well-structured essays. And I'll be honest – I used to scroll straight to the last paragraph where it summarises everything. That's the part I actually needed. I now have rules in place to change this behaviour, but the verbosity problem is real and it costs you – in time, in tokens, in API bills.

Caveman takes a completely different approach: constrain the output style at the source. Make the agent respond in stripped-down, primitive language patterns. Short words. No fluff.

The numbers:

- **~75% reduction in output tokens** – structural cost and speed win, not a rounding error
- Preserves meaning while gutting verbosity
- Targets the problem at generation, not post-processing

*"More human-like" doesn't always mean better for agent output.
Sometimes the right output style for a machine is deliberately minimal.*

Fun fact: Kevin Malone from The Office was running this mode his entire career – *"Why waste syllable when few do trick?"* Turns out Kevin wasn't bad at communication. He was just ahead of his time on token efficiency.

For internal agent pipelines especially, this is worth a weekend experiment.

 BYLINE · PERSONAL

My dev setup, now with Claude.

Anthropic just released a redesigned Claude Desktop app – and honestly, this one's personal.

I've been running Warp terminal for a while now. The reason? It gave me a terminal, a file viewer, and a diff viewer all in one place. That's it. That's the whole reason. Switching between tools mid-flow is where I lose the thread, and Warp solved that for me.

Claude Desktop's new release does the same thing:

- **Integrated terminal** – right there in the app
- **In-app file editing** – open and edit without jumping out
- **Rebuilt diff viewer** – see what changed, properly rendered
- **Side chats** – parallel conversations for parallel work
- **SSH connections** – into remote environments directly from the app

Glad Anthropic shipped this. It's the setup I already wanted, now built into the tool I'm already using most.

If you're a developer and you haven't updated yet – worth doing today.

↓ INCOMING · PATTERN

The advisor-executor *pattern*.

*Anthropic just made something quietly clever available on the Claude Platform: the **advisor-executor pattern** – and I think it's one of the more practical agent architecture ideas to land in a while.*

Here's how it works:

- Pair **Claude Opus as an advisor** with Sonnet or Haiku as the executor
- When the executor hits a hard decision mid-task, it can consult Opus for a plan
- Opus responds, the executor picks back up – all within a **single API request**
- In evals, Sonnet + Opus advisor scored **2.7 points higher on SWE-bench Multilingual** than Sonnet alone
- Cost? **11.9% cheaper per task** than running Opus solo

The big idea: you don't need Opus running the whole show – you just need it available at the right moments.

This is how senior engineers actually work. You don't pull in your most expensive expert for every line of code – you call them when the decision actually matters. Anthropic is baking that pattern into the API itself.

The oracle is only as good as the question you know to ask – and deciding when to escalate will likely become its own engineering discipline.

TRENDING · SAFETY

LangWatch just open-sourced *adversarial testing*.

The LangWatch team just open-sourced an automated red-teaming tool for AI agents – announced by Manouk Draisma – and it addresses something most teams are quietly ignoring.

The post opens with a sharp observation: turn on the news and AI sounds terrifying – deepfakes, voice cloning, disinformation, data authorities calling agents "Trojan horses." Those concerns are real. But the harder and more immediate question is whether the AI systems we're building can actually hold up under deliberate attack.

The specific problem: attacks on AI agents don't look like one hostile prompt. They look like a patient, escalating conversation – building trust over multiple turns, probing edges quietly – until the agent gives up something it shouldn't. Most test suites aren't built for this. They're built for happy paths and obvious failures.

That's the gap **LangWatch Red-teaming** is designed to close:

- **Multi-turn attack simulation** – models the slow crescendo of a real adversarial conversation, not just isolated edge cases
- **Crescendo escalation** – a specific technique that mirrors how real-world manipulation unfolds over time, gradually pushing the agent past its guardrails
- **Automated adversarial testing at scale** – not manual edge case writing, but systematic pressure across the attack surface
- **CI/CD integration** – plugs into your pipeline so safety testing is continuous, not a one-time pre-launch audit
- One method call to get started

The gap between "passes all our tests" and "safe in production" is real, and most teams are not closing it. Testing agentic systems at the unit or integration level is something the industry is still largely winging.

This is open source and part of LangWatch Scenario. If you're building anything with AI agents that touches customer data, internal systems, or complex workflows – this is a serious place to start.

REFLECT · A SLOWER READ

The names behind *the AI incidents*.

A 14-year-old boy's last words were to an AI chatbot. Not to his parents — they were home when he died.

His name was Sewell Setzer III. Juliana Peralta, 13. A 17-year-old in Texas who ended up in inpatient care after a chatbot suggested harming his parents.

These aren't hypothetical risks. They are children.

WHAT THE CHARACTER.AI CASE REVEALED

- The company allegedly knew chatbots could cause depression and suicidal ideation — and didn't disclose it
- When minors said they wanted to end their lives, the bot kept engaging
- Children as young as 9 were exposed to sexually explicit conversations
- The FTC inquiry came in September 2025. The age ban followed in October — *after the deaths*
- AI incidents rose 50% year-over-year from 2022–2024. 2025 alone: 346 incidents, already past the full 2024 total

Google acquired Character.AI's founders for \$2.7B. Lawsuits settled in January 2026. Families got compensation. The children didn't come back.

This is the part of the AI story I can't look away from. The industry celebrated the upside and moved on to the next product launch. But the downside has names. And empty bedrooms.

Are our research labs truly capable of facing catastrophic accidents like these? Not just technically — but with the accountability and urgency that matches the stakes?

A CALL TO ACTION

Pause, reflect, and initiate conversations. Collaborate with AI governance experts. Let's push for a community-driven AI landscape.

A curated dump of interesting posts — synthesised for easy reading.

Tech Digest with Sam is a curated dump of interesting posts I came across on LinkedIn this week — synthesised and written up for easy reading.

Content referencing third-party posts remains the copyright of their respective authors. This newsletter is circulated for informational purposes only.



Sam Kirubaharan *(Sam KM)*

AI-FIRST LEAD ENGINEER

WEB ENTHUSIAST · CUSTOMER SUPPORT TECH SPECIALIST

I'm an AI-first Lead Engineer with nearly a decade of experience in building software applications on Cloud-native, Mobile-first and Web-centered solutions.

LINKEDIN [linkedin.com/in/sam-kirubaharan-5973b8110](https://www.linkedin.com/in/sam-kirubaharan-5973b8110)

MEDIUM samkmbuilds.medium.com



LOVE



TREND



REFLECT



IN



CURIOUS



BYLINE